Minor Project Report

API Rate Limiting System

Project Category: Deep Tech Based

Projexa Team Id-26E3179

Submitted in partial fulfilment of the requirement of the degree of

BACHELOR OF SCIENCE(Hons) COMPUTER SCIENCE

Semester-IV

to

K.R Mangalam University

by

Sidharth Krishna S (2401720003)
Ankit Yadav (2401720012)
Palak Kashyap (2401840007)
Diwakar (2401720013)
Devasya Dahiya (240172001)

Under the supervision of
Dr. Ravinder Beniwal
Assistant Professor

Department of Computer Science and Engineering
School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

May 2026

DECALARATION

I hereby declare that this project report is my original work and has not been submitted elsewhere. The information presented here is true and correct to the best of my knowledge.

(Signature)

Student Name: Sidharth Krishna S

Student Roll No: 2401720003

Student Name: Ankit Yadav

Student Roll No: 2401720012

Student Name: Palak Kashyap

Student Roll No: 2401840007

Student Name: Diwakar

Student Roll No: 2401720013

Student Name: Devasya Dahiya

Student Roll No: 2401720001

Date 10/04/26

CERTIFICATE

This is to certify that the Project Synopsis entitled, "**API DISTRIBUTED RATE LIMITER SYSTEM**" submitted by "**Sidharth Krishna S(2401720003) , Ankit Yadav(2401720012),Palak(2401840007), Diwakar(2401720013) and Devasya Dahiya(2401720001)**" to **K.R Mangalam University, Gurugram, India,** is a record of Bonafide project work carried out by them under my supervision and guidance and is worthy of consideration for the partial fulfilment of the degree of **Bachelor of Science** in **Computer Science** of the University.

10/04/2026

Dr. Ravinder Beniwal
Assistant Professor
SOET

Date: 20th April 2026

INDEX

S No	Content	Page No
1.	Project Completion Certificate	
2.	Abstract	
3.	Introduction (description of broad topic)	
4.	Motivation	
5.	Literature Review/Comparative work evaluation	
6.	Gap Analysis	
7.	Problem Statement	
8.	Objectives	
9.	Tools/platform Used	
10.	Methodology	
11.	Experimental Setup	
12.	Evaluation Metrics	
13.	Results And Discussion	
14.	Conclusion & Future Work	
15.	References	
16.	Annexure I: Plagiarism Report	
17.	Annexure II Complete implementation code (optional)	

ABSTRACT

Modern web applications require efficient API traffic control to ensure stability, prevent abuse, and maintain fair usage. This project presents a Distributed Rate Limiter System that regulates API requests in a scalable and reliable manner.The system is developed using Spring Boot, Redis, and PostgreSQL. It uses the Sliding Window Algorithm to control request flow by tracking requests within a moving time window, ensuring smooth and accurate rate limiting while reducing sudden traffic spikes.

An API Key Management System allows administrators to generate and manage API keys with configurable limits, while an Admin Authentication System secures access to critical features.

For monitoring, Prometheus and Grafana are used to track real-time metrics such as request rates and system performance. The application is containerized using Docker for consistent deployment and scalability.

Overall, the system provides an effective solution for managing API traffic in environments like API gateways and microservices architectures.

KEYWORDS:

- ***Distributed Rate Limiting***
- ***Sliding Window Algorithm***
- ***API Key Management***
- ***Redis, Spring Boot***
- ***PostgreSQL***
- ***Prometheus Monitoring***
- ***Docker Containerization***
- ***Grafana Visualization***

Chapter 1
Introduction

1. Background of the project

Over the past decade, web applications, cloud platforms, and microservices architectures have grown significantly, with APIs playing a central role in enabling communication between systems. Every user interaction or service-to-service communication relies heavily on APIs. In many scenarios, systems must handle thousands of requests simultaneously, which can lead to server overload, resource exhaustion, and potential security threats such as denial-of-service (DoS) attacks. Therefore, controlling the rate of incoming requests has become essential for maintaining system stability and reliability.

However, traditional systems were not designed to handle such high levels of traffic efficiently. In the absence of proper rate limiting mechanisms, some users may consume excessive resources while others experience degraded performance or service unavailability. This issue becomes more complex in distributed environments, were multiple servers process requests concurrently. In such cases, relying on a centralized rate limiting approach is not scalable or efficient.

To overcome these challenges, distributed rate limiting techniques are used, where request limits are enforced across multiple servers. Technologies like Redis play a crucial role by storing request data in memory, enabling fast and efficient rate limit evaluation. Additionally, frameworks such as Spring Boot provide a scalable and reliable foundation for building high-performance backend systems.

Various algorithms are used for rate limiting, including Token Bucket, Leaky Bucket, and Sliding Window. Among these, the Sliding Window Algorithm is preferred for its ability to provide more accurate and smooth traffic control by considering request patterns over a continuous time interval.

Modern systems also require real-time monitoring and observability. Tools like Prometheus and Grafana are widely used to collect and visualize system metrics, helping administrators analyse performance and detect anomalies. Furthermore, Docker ensures consistent deployment by containerizing applications and their dependencies.

This project addresses these challenges by designing and implementing a distributed rate limiting system that effectively controls API traffic, prevents system overload, and ensures scalability, security, and efficient performance.

Distributed Rate Limiter System

Factors	Evaluation Criteria	System A (Sliding Window)	System B (Token Bucket)	System C (Leaky Bucket)
Rate Limiting Accuracy	- Precision in controlling requests	High	Moderate	Moderate
	- Fair distribution of requests	Excellent	Good	Limited
Traffic Handling	- Burst traffic handling	Smooth handling	Excellent (handles bursts)	Poor (strict flow)
	- Sudden spike control	Very Effective	Moderate	Effective but rigid
Performance	- Response time	Fast	Fast	Moderate
	- Throughput under high load	High	High	Moderate
Scalability	- Suitability for distributed systems	Highly Scalable	Scalable	Less Scalable
	- Integration with Redis	Seamless	Possible	Limited
Implementation Complexity	- Ease of implementation	Moderate	Easy	Low
Real-time Monitoring	▸ Fully Compatible	Fully Compatible	Fully Compatible	Fully Compatible
Use Case Suitability	- Best for modern APIs	✓ Highly Suitable	Suitable	Less Suitable
Use Case Suitability	- Best for modern APIs	Highly Suitable	Suitable	Less Suitable
	- Fairness in API usage	✓ Excellent	Good	Limited

2.MOTIVATION

In today's digital era, the rapid growth of web applications, cloud computing, and microservices has increased reliance on APIs for system communication. As API usage grows, managing high volumes of traffic becomes a critical challenge for maintaining system performance and reliability.

One major motivation for this project is to prevent system overload and ensure fair resource usage. Without proper rate limiting, excessive or malicious requests can degrade performance or cause system failures, especially in critical applications like payment systems and large-scale platforms.

Another key motivation is to overcome the limitations of traditional, centralized rate limiting approaches. Modern distributed systems require scalable solutions that can enforce limits consistently across multiple servers. This project addresses this by implementing a distributed rate limiting system.

Technologies like Redis enable fast request tracking, while the Sliding Window Algorithm ensures accurate and fair control of request flow. Additionally, integrating Prometheus and Grafana allows real-time monitoring and better system management.

The use of Docker further supports scalability and consistent deployment across environments.

Overall, this project is motivated by the need to build a robust, scalable, and efficient solution for managing API traffic in modern distributed systems

Chapter 2

LITERATURE REVIEW

1. Review of existing literature

With the increasing reliance on web services and APIs, rate limiting has become essential for ensuring system stability, security, and fair resource utilization. Various approaches have been developed, each with its own strengths and limitations.

The Token Bucket Algorithm allows burst traffic while maintaining an average request rate, but it may lead to uneven distribution. The Leaky Bucket Algorithm ensures a steady flow of requests, though it lacks flexibility in handling sudden traffic spikes.

More advanced methods like the Sliding Window Algorithm provide more accurate and fairer rate limiting by analysing requests over a continuous time window. This makes it more suitable for modern applications with dynamic traffic patterns.

In distributed systems, maintaining consistency and low latency is a major challenge. To address this, in-memory data stores like Redis are widely used for fast and scalable request tracking.

Monitoring tools such as Prometheus and Grafana help in analyzing real-time system performance, while Docker enables consistent deployment and scalability across environments.

Despite these advancements, many existing solutions lack scalability or accuracy in distributed environments. This project addresses these gaps by combining the Sliding Window Algorithm with Redis, along with modern monitoring and deployment tools, to provide a robust and efficient rate limiting solution.

Table 2. LITERATURE REVIEW/COMPARITIVE WORK

Project Title	Objectives	Technologies Used	Outcomes and Findings
API Rate Limiting using Token Bucket Algorithm	To control API traffic and allow burst requests while maintaining average rate	Java, Spring Boot, Token Bucket Algorithm	Efficient for burst traffic handling but lacks fairness and may cause uneven request distribution
Leaky Bucket Based Traffic Control System	To ensure a steady and controlled flow of API requests	Python, Flask, Leaky Bucket Algorithm	Provides consistent output rate but cannot handle sudden traffic spikes effectively
Sliding Window Rate Limiter for Distributed Systems	To improve accuracy and fairness in request handling across distributed environments	Node.js, Redis, Sliding Window Algorithm	Offers better fairness and smooth traffic control, suitable for modern scalable systems
Redis-Based Distributed Rate Limiting System	To implement high-performance and scalable rate limiting using in-memory storage	Spring Boot, Redis, Docker	Achieves low latency and high scalability, widely used in real-world applications
API Monitoring and Rate Control System with Observability	To combine rate limiting with real-time monitoring and performance analysis	Prometheus, Grafana, Docker, Microservices	Enhances system visibility and helps in detecting anomalies, improving overall system reliability

2.GAP ANALYSIS

Existing research on API rate limiting has introduced various algorithms such as Token Bucket, Leaky Bucket, and Fixed Window techniques to control traffic and prevent system overload. These approaches are effective in basic scenarios but often face limitations in modern distributed environments.

Many existing solutions rely on centralized architectures, which can lead to scalability issues and inconsistent rate enforcement across multiple servers. Additionally, traditional algorithms may not provide accurate or fair request handling, especially under dynamic and high-traffic conditions.

Another major gap is the lack of integration between rate limiting mechanisms and real-time monitoring systems. Many implementations do not offer proper visibility into request patterns, blocked traffic, or system performance, making it difficult to optimize and manage the system effectively.

This project addresses these gaps by implementing a distributed rate limiting system using the Sliding Window Algorithm, which ensures more accurate and fair control of API requests. It integrates Redis for fast and scalable request tracking, along with Prometheus and Grafana for real-time monitoring and analysis. Docker is used to ensure consistent deployment and scalability across environments.

Overall, the proposed system provides a more scalable, efficient, and observable solution compared to traditional rate limiting approaches, making it suitable for modern API-driven applications.

3.PROBLEM STATEMENT

Application Programming Interfaces (APIs) are a core component of modern web applications, enabling communication between different services and systems. With the rapid growth of web platforms and microservices architectures, APIs are exposed to a large volume of requests, making them vulnerable to overuse, misuse, and malicious attacks.

Traditional systems often lack effective mechanisms to control and regulate API traffic. Without proper rate limiting, excessive requests can overload servers, degrade performance, and even lead to system failures. Additionally, many existing rate limiting solutions rely on centralized architectures, which are not suitable for distributed environments and may result in inconsistent enforcement of request limits.

Another major limitation is the lack of accurate traffic control and real-time monitoring. Conventional approaches may not handle dynamic traffic patterns efficiently and often fail to provide visibility into system performance, blocked requests, and usage patterns. This makes it difficult for administrators to manage and optimize system behaviour.

These challenges create the need for a scalable and efficient rate limiting system that can handle high traffic loads, ensure fair usage, and provide real-time insights. Such a system should be capable of operating in distributed environments, maintaining consistency across multiple servers, and enhancing overall system reliability and security.

4.OBJECTIVES

The features include:

- **Request Monitoring** – Monitors incoming API requests in real time.
- **Rate Limiting** – Controls the number of requests using the Sliding Window Algorithm to ensure fair usage.
- **API Key Management** – Generates and manages API keys with configurable rate limits.
- **Authentication System** – Secures access to admin functionalities and sensitive endpoints.
- **Traffic Analysis** – Tracks allowed and blocked requests for better decision-making.

The objective of this project is to overcome the limitations of traditional rate limiting systems and develop a scalable, distributed solution for managing API traffic. The system ensures efficient request handling, prevents server overload, and maintains fairness among users.

By integrating technologies like Redis for fast data handling, Prometheus and Grafana for monitoring, and Docker for deployment, the project provides a reliable and production-ready solution. Additionally, the system is designed to be efficient, easy to manage, and suitable for modern web applications and microservices architectures.

CHAPTER 3: METHODOLOGY

The methodology section describes the design, implementation, and working of the proposed system. It explains how different components interact to achieve efficient and scalable API rate limiting.

3.1 **Overall architecture /Flow chart:**

The overall architecture represents the high-level structure of the system, showing how different modules communicate and function together. The system follows a distributed architecture to ensure scalability, reliability, and consistent rate limiting across multiple services.

Main Components:

- **Client Layer** – Sends API requests to the server.
- **API Gateway / Application Server (Spring Boot)** – Receives and processes incoming requests.
- **Rate Limiter Module** – Applies the Sliding Window Algorithm to decide whether a request should be allowed or blocked.
- **Redis (In-Memory Store)** – Stores request timestamps and counters for fast access and real-time decision-making.
- **PostgreSQL Database** – Stores API keys, user data, and configuration details.
- **Monitoring System (Prometheus & Grafana)** – Collects and visualizes system metrics.
- **Docker Environment** – Ensures consistent deployment and scalability.

Working Flow:

1. Client sends an API request.
2. Request is received by the Spring Boot server.
3. The Rate Limiter checks request history in Redis.
4. Sliding Window Algorithm evaluates if the request is within the allowed limit.
5. If allowed → request is processed.
6. If exceeded → request is blocked.
7. All request data is monitored and visualized using Prometheus and Grafana.

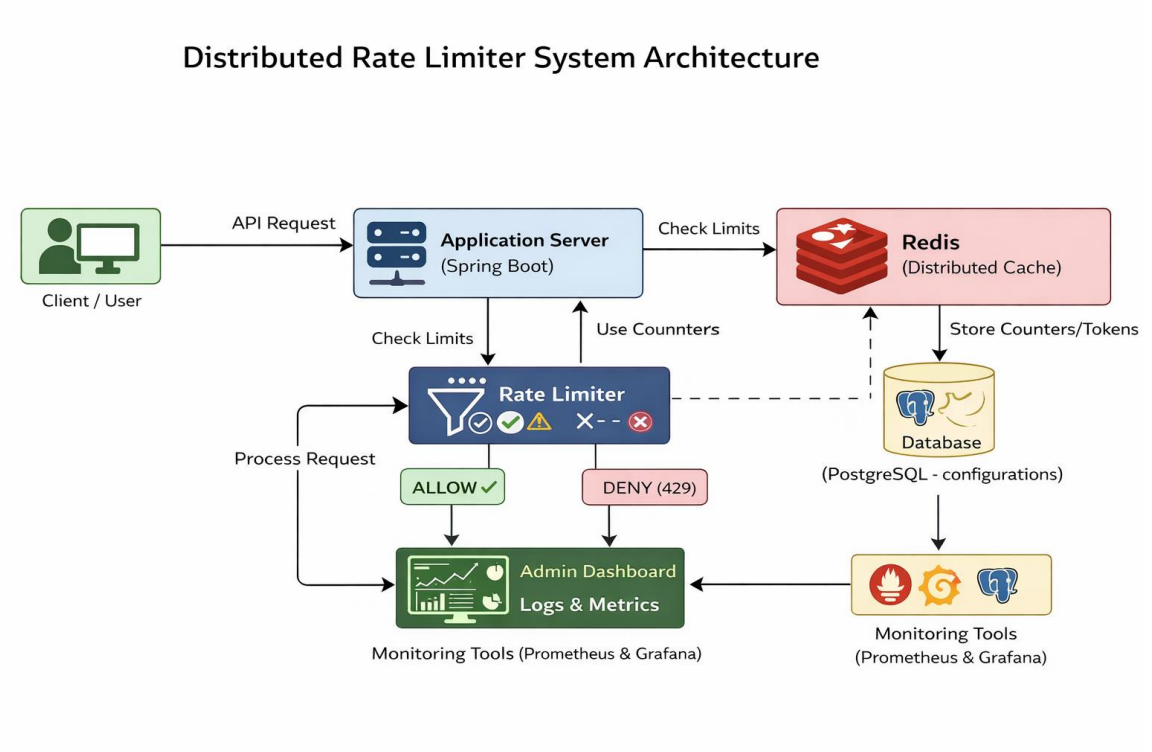


Figure 1. System Architecture Diagram

3.2 Data Description

Data Source:
The data used in this project is generated dynamically from API requests made by clients. It includes request logs, timestamps, API keys, and response status (allowed/blocked). These are stored using Redis for real-time processing and PostgreSQL for persistence.

Data Collection Process:

Data is collected automatically when a client sends an API request.

- Each request is intercepted by the server
- Timestamp and API key are recorded
- Data is stored in Redis for rate limiting decisions
- Important details are stored in PostgreSQL

Data Type:

- Time-series data (request timestamps)
- Categorical data (API keys, request status)
- Numerical data (request count, limits)

Data Size:
The data size depends on traffic volume. The system is designed to handle high-frequency API requests in real time, scaling dynamically with usage.

Data Format:

- Redis: Key-value format
- PostgreSQL: Structured tables
- JSON: Used for API communication

Data Preprocessing:

- Filtering invalid requests
- Removing expired timestamps (sliding window cleanup)
- Normalizing request logs

Data Quality Assurance:

- Validation of API keys
- Rate limit checks
- Error handling for invalid or duplicate requests

Data Variables:

- API Key
- Timestamp
- Request Count
- Rate Limit Threshold
- Request Status (Allowed/Blocked)

Data Distribution and Summary Statistics:

The data in this project is generated from API requests and mainly includes request timestamps, request count, and request status (allowed or blocked). Basic statistics such as average request rate, maximum requests, and variation in traffic are used to understand system performance. These help in identifying normal usage patterns and sudden traffic spikes. Instead of static graphs like histograms, real-time visualization is done using Grafana dashboards. These show request trends and system behaviour over time, making it easier to monitor and analyse the data.

3.3Development Life Cycle

- **Requirement Analysis:**
  The need for a scalable API rate limiting system was identified to prevent server overload, ensure fair usage, and handle high traffic in distributed environments. Key requirements included request monitoring, rate limiting, API key management, and real-time performance tracking.

- **System Design:**
  A distributed architecture was designed using Spring Boot for backend development, Redis for fast in-memory data storage, and PostgreSQL for persistent storage. The design ensures consistent rate limiting across multiple instances.

- **Algorithm Implementation:**
  The Sliding Window Algorithm was implemented to control API traffic. It tracks request timestamps within a moving time window, removes expired entries, and determines whether a request should be allowed or blocked, ensuring accurate and fair rate limiting.

- **Backend Development | Database Integration:**
  REST APIs were developed using Spring Boot, and Redis was integrated for fast request handling. PostgreSQL was used to store API keys and configuration data, ensuring structured and secure data management.

- **Monitoring | Containerization:**
  Prometheus and Grafana were integrated for real-time monitoring and visualization of system metrics. Docker was used for containerization, enabling consistent deployment and easy scalability.

- **Testing | Validation:**
  The system was tested under different traffic conditions to verify performance, accuracy, and reliability. Various scenarios, including high traffic and burst requests, were evaluated to ensure robustness.

1.Details of tools, software, and equipment utilized.

PLATFORM USED

For this project, modern technologies are used to build a scalable and efficient distributed rate limiting system. Each technology is selected based on performance, scalability, and ease of integration.

PROGRAMMING LANGUAGE: **JAVA (SPRING BOOT)**

Java is used as the primary programming language due to its reliability, scalability, and strong support for building enterprise-level applications. The project is developed using Spring Boot, which simplifies backend development by providing built-in features for creating REST APIs and managing application configurations.

Spring Boot enables faster development, easy integration with databases, and supports microservices architecture, making it suitable for building distributed systems like a rate limiter. It also ensures better performance and maintainability compared to traditional development approaches.

Reasons for Selecting this language:

1. **Platform Independent** – Java runs on any system using the Java Virtual Machine (JVM).
2. **Robust and Scalable** – Suitable for building large-scale and distributed applications.
3. **Strong Framework Support** – Spring Boot simplifies backend development and API creation.
4. **Object-Oriented Programming** – Promotes modular, reusable, and maintainable code.
5. **High Performance** – Efficient handling of concurrent requests, important for rate limiting systems.

Some specific features of Java are as follows:

1. **Compiled and Platform Independent** – Java code is compiled into bytecode and runs on the Java Virtual Machine (JVM), allowing it to work on any platform.
2. **Open Source and Widely Supported** – Java is freely available and supported by a large developer community, making it reliable for both academic and commercial applications.
3. **Multi-platform Compatibility** – Java applications can run on Windows, Linux, macOS, and other operating systems without modification.
4. **Readable and Structured Syntax** – Java provides a clear and structured syntax, making code easy to understand and maintain.
5. **Rich Ecosystem and Libraries** – A wide range of libraries and frameworks like Spring Boot are available for building web applications and backend systems.
6. **Easy Integration** – Java can easily integrate with databases, APIs, and other technologies such as Redis and Docker.
7. **Object-Oriented Programming** – Supports concepts like encapsulation, inheritance, and polymorphism, ensuring modular and reusable code.
8. **High Performance and Reliability** – Suitable for handling large-scale and high-concurrency applications like distributed systems.

The features included in the project are:

1. **Request Monitoring** – Monitors incoming API requests in real time.
2. **Rate Limiting** – Controls the number of requests using the Sliding Window Algorithm.
3. **API Key Management** – Generates and manages API keys with configurable limits.
4. **Authentication** – Secures access to admin and system endpoints.
5. **Traffic Analysis** – Tracks allowed and blocked requests for better system insights. Each feature is discussed in detail in the next section, the methodology used in the features is elaborated and the process model used for the completion of the project is also mentioned below.

2.ENVIRONMENTAL SETUP

SOFTWARE REQUIREMENTS

Below are the requirements to run this software:

1. Windows / Linux / macOS (any version)
2. Java (JDK 8 or above)
3. Spring Boot Framework
4. Redis Server
5. PostgreSQL Database
6. Docker (for containerization)
7. Prometheus and Grafana (for monitoring)
8. Any API testing tool (Postman / Curl)

HARDWARE REQUIREMENTS

In terms of hardware requirements there is not much required at all but still below requirements are must:

1. A working PC or Laptop
2. Minimum 4GB RAM (8GB recommended)
3. Stable internet connection

PLATFORMS ALREADY TESTED ON:

- Windows 10 / 11

Chapter 4

Implementation

System Implementation

The Distributed Rate Limiter System is implemented using a backend developed in Spring Boot, which handles API requests and applies rate limiting logic. The system uses Redis as an in-memory data store to maintain request timestamps and counters, enabling fast and efficient processing. PostgreSQL is used to store API keys, user details, and configuration data. When a client sends a request, it is first received by the server, where the rate limiter module checks the request history from Redis. Based on the Sliding Window Algorithm, the system decides whether the request should be allowed or blocked. If the request is within the defined limit, it is processed; otherwise, it is rejected.

Algorithm Used: Sliding Window Algorithm

The Sliding Window Algorithm works by tracking all requests within a specific time window. Instead of resetting counters at fixed intervals, it continuously updates the window by removing expired requests and adding new ones.

Steps:

- Store request timestamps in Redis
- Remove timestamps older than the defined time window
- Count the number of valid requests
- If count exceeds limit → block request
- Else → allow request

Design Overview:

- Client sends request
- Spring Boot processes request
- Redis stores and retrieves request data
- Rate limiter applies decision
- PostgreSQL manages API keys
- Prometheus and Grafana monitor performance

Challenges and Solutions:

Handling High Traffic:
Large volumes of requests can slow down processing.
Solution: Used Redis for fast in-memory operations.
Maintaining Accuracy:
Traditional algorithms caused uneven rate limiting.
Solution: Implemented Sliding Window Algorithm for better accuracy.
Scalability Issues:
Centralized systems are difficult to scale.
Solution: Designed a distributed architecture with Docker.
Monitoring System Performance:
Difficult to track real-time metrics.
Solution: Integrated Prometheus and Grafana for monitoring.

RESULTS AND DISCUSSION

The implemented Distributed Rate Limiter System was tested under different traffic conditions to evaluate its performance and effectiveness. The system successfully controlled API request rates and prevented overload by blocking requests that exceeded the defined limits.

The Sliding Window Algorithm provided accurate and smooth rate limiting compared to traditional methods, ensuring fair usage among clients. Redis enabled fast processing of requests, resulting in low latency even under high traffic conditions.

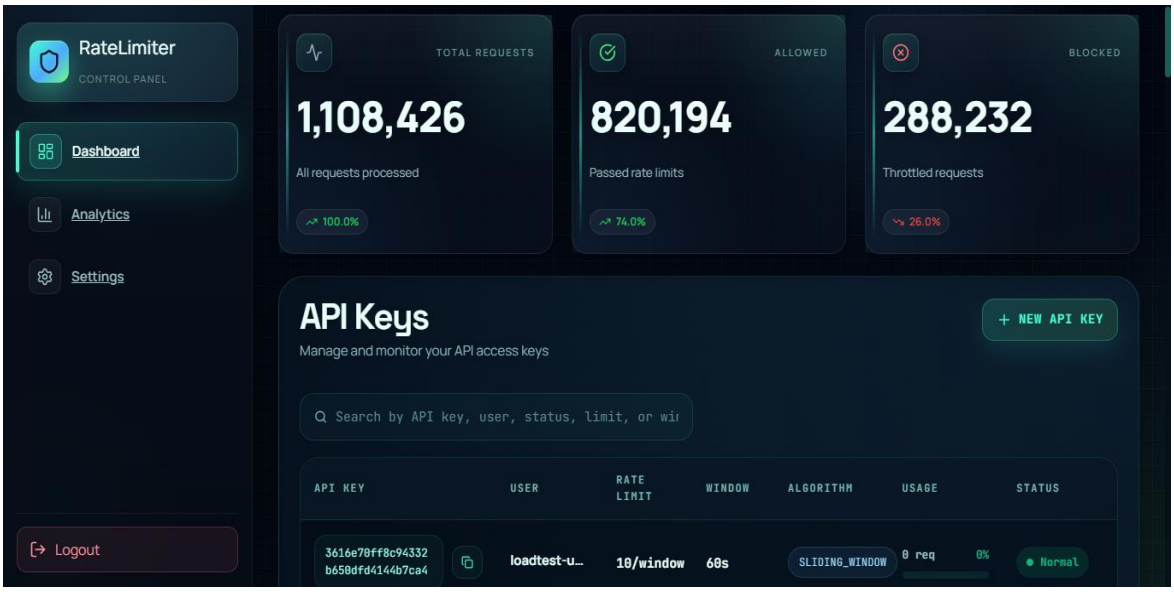
Monitoring tools such as Prometheus and Grafana were used to visualize system performance. Metrics like total requests, allowed requests, and blocked requests were tracked in real time. These insights helped in understanding traffic patterns and optimizing system behaviour.

The system demonstrated good scalability when deployed using Docker, allowing it to handle increasing loads efficiently. Overall, the results show that the proposed system is reliable, efficient, and suitable for real-world applications such as API gateways and microservices environments.

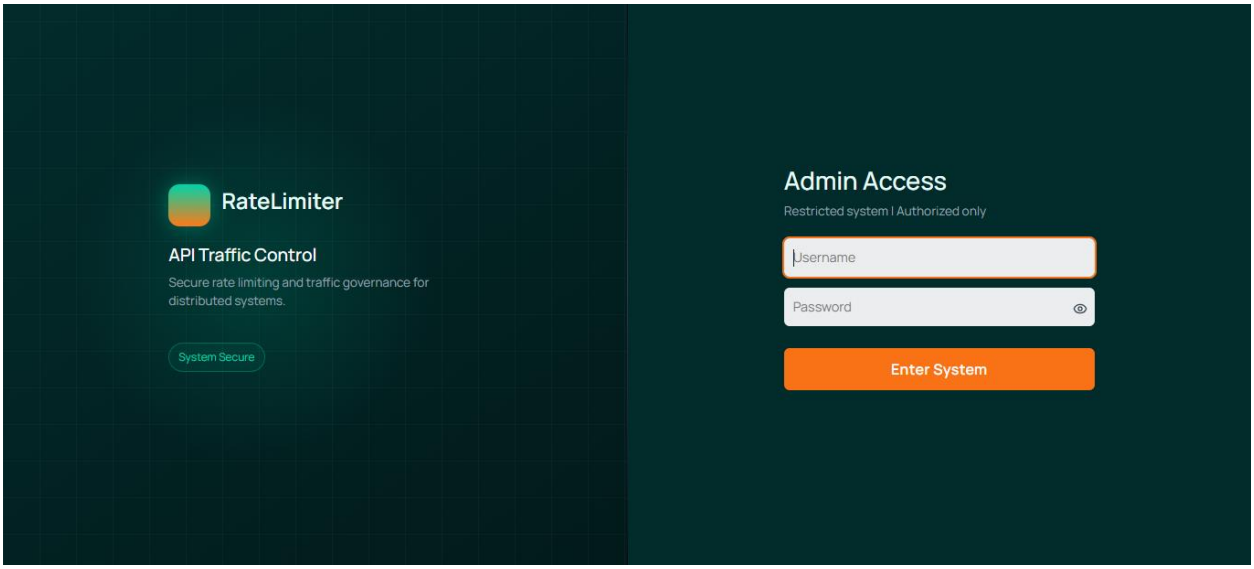
Chapter 5

RESULTS AND DISCUSSIONS

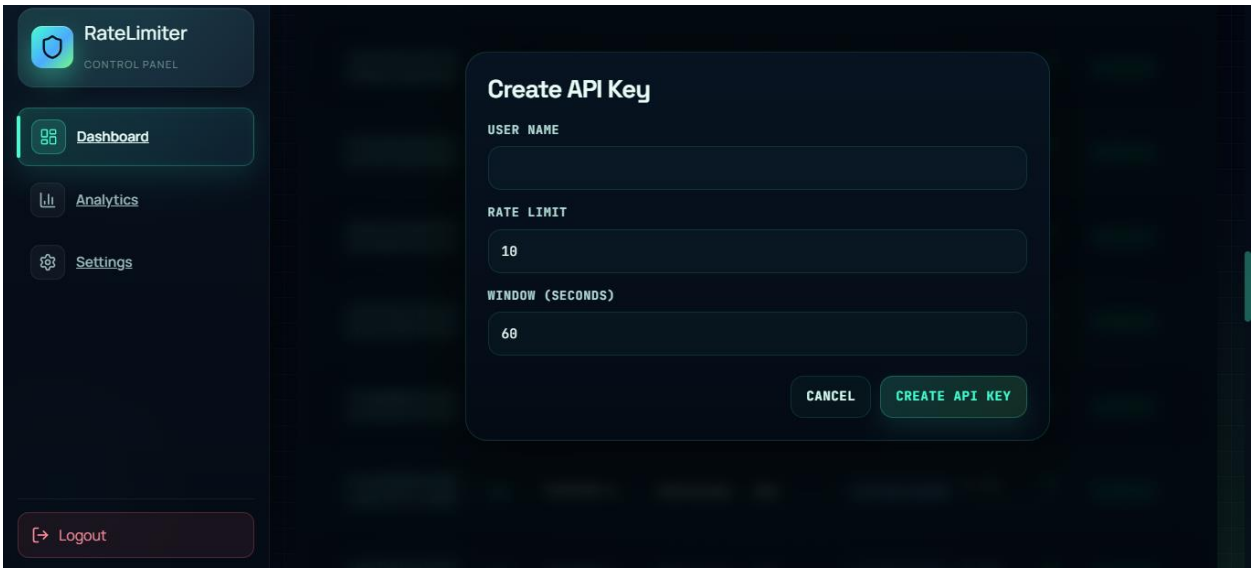
1. Dashboard



2.Login Page

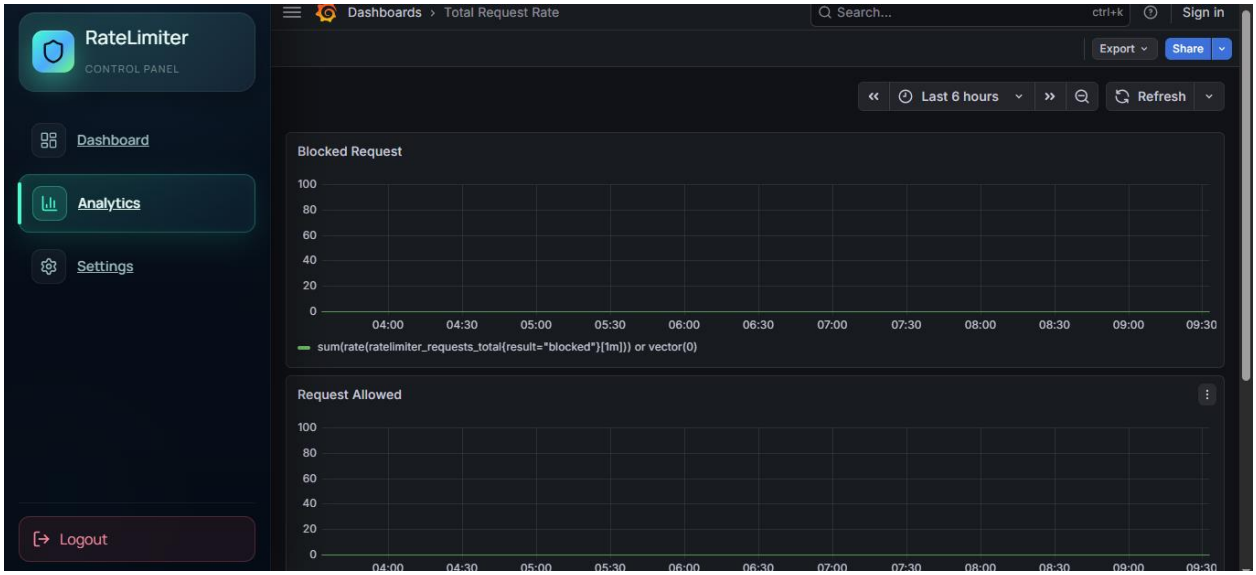


3. API key creation model

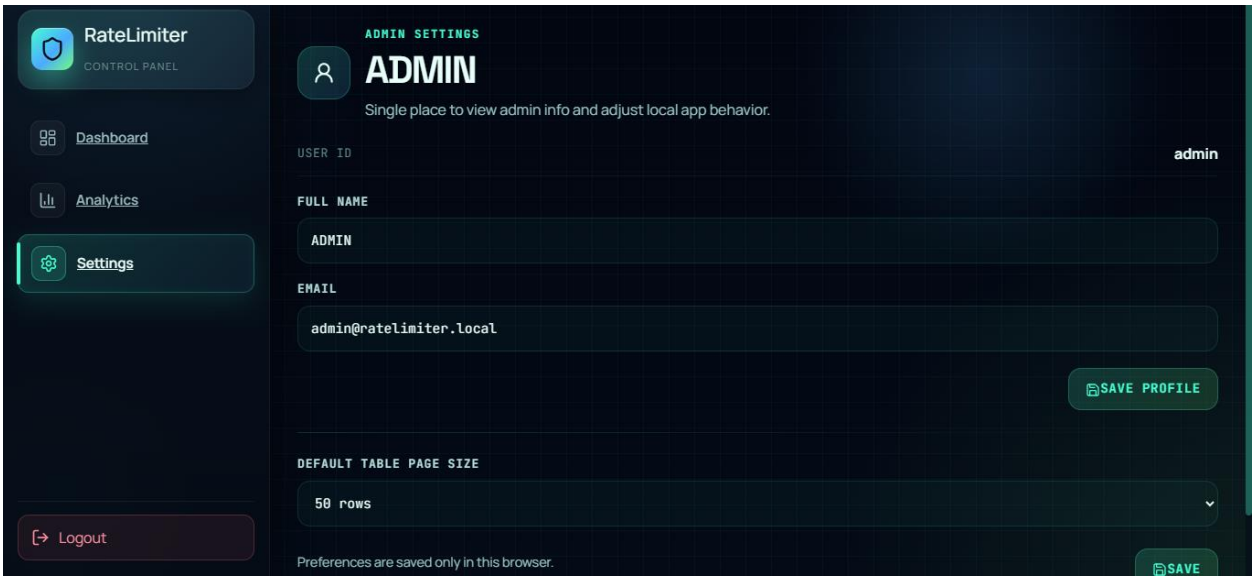


4. API keys table

5.Analytics_Page



5. Settings Page



Chapter 6

FUTURE WORK

The proposed Distributed Rate Limiter System provides an effective solution for controlling API traffic; however, it can be further improved in several ways. One important enhancement is the use of **adaptive rate limiting**, where limits can automatically adjust based on traffic patterns and system load instead of remaining fixed.The system can also be extended to support **multi-region deployment**, ensuring consistent rate limiting across geographically distributed servers. This would improve scalability and performance for large-scale applications.Another area of improvement is the development of a **custom monitoring dashboard** that provides more detailed insights and better visualization of system performance and API usage.In addition, **security features** such as anomaly detection and protection against malicious requests can be integrated to make the system more robust.Finally, implementing **high availability mechanisms**, such as Redis clustering and failover support, can further enhance system reliability and ensure continuous operation in case of failures.

CONCLUSION

In modern web applications, the rapid growth of APIs and distributed systems has increased the need for efficient traffic management and system reliability. Without proper control mechanisms, excessive or malicious requests can lead to server overload, degraded performance, and system failures.

This project presents a Distributed Rate Limiter System designed to address these challenges by effectively regulating API traffic. The implementation of the Sliding Window Algorithm ensures accurate and fair rate limiting by dynamically tracking request patterns, reducing sudden spikes, and improving overall system stability.

The use of technologies such as Spring Boot, Redis, and PostgreSQL enables efficient request handling, fast data access, and reliable storage. Additionally, integrating Prometheus and Grafana provides real-time monitoring and visualization of system performance, helping administrators make informed decisions. Docker further enhances the system by ensuring consistent deployment and scalability across different environments.

Overall, the system successfully prevents API abuse, ensures fair resource utilization, and maintains high performance under varying traffic conditions. It provides a scalable and reliable solution suitable for real-world applications such as API gateways and microservices architectures, contributing to improved system security and efficiency.

REFERENCES

[1] Kleppmann M. (2017) Designing Data-Intensive Applications. O'Reilly Media. Reference for distributed systems, scalability, and consistency principles.

[2] Redis Documentation Redis: In-Memory Data Structure Store. https://redis.io/docs/ Reference for centralized counters, atomic operations, and low-latency data access.

[3] Amazon Web Services (AWS) API Gateway – Request Throttling and Rate Limiting. https://docs.aws.amazon.com/apigateway/ Industry reference for real-world API rate limiting and traffic control.

[4] NGINX Documentation. Rate Limiting and Traffic Control. https://docs.nginx.com/nginx/admin-guide/security-controls/controllingaccess/ Practical implementation reference for rate limiting algorithms and request control.

ANNEXURE I: Plagiarism Declaration Certificate

Title of Work: Distributed API Rate Limiting System
Submitted By: Sidharth Krishna S (2401720003), Ankit Yadav (2401720012), Palak Kashyap (2401840007), Diwakar (2401720013), Devasya Dahiya (2401720001).
Institution: K. R. Mangalam University, Gurugram
Department: SOET
Date of Submission: 05-04-2026

I hereby declare that the work entitled submitted for academic evaluation and research purposes is my original work. I confirm that:

- I have acknowledged and properly cited all sources, references, and data included in this work.
- This work does not contain any material previously published, written, or prepared by another person, except where due acknowledgment has been made.
- I understand that plagiarism is an academic offense and a violation of research ethics. Any breach of this declaration may lead to disciplinary action as per university policies.
- I have used appropriate referencing techniques and maintained academic integrity throughout this work.

I affirm that the submitted work has normal plagiarism less than 10% and free from AI content.

Signature of Students: Sidharth 10/04/26 Ankit 10/04/26 Palak Kashyap 10/04/26 Diwakar 10/04/26

Date: 10/04/26

Devasya 10/04/26

Annexure II: Complete implementation code

Entity Layer

```java
@Entity
@Table(name = "api_keys")
public class ApiKey {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "user_name", nullable = false)
    private String userName;

    @Column(nullable = false)
    private Integer rateLimit;

    @Column(nullable = false)
    private Integer windowSeconds;

    @Column(nullable = false, columnDefinition = "varchar(30) default 'SLIDING_WINDOW'")
    private String algorithm;

    @Column(name = "api_key", nullable = false)
    private String apiKey;

    @Column(nullable = false, columnDefinition = "varchar(20) default 'Normal'")
    private String status;

```

Repository Layer

```java
public interface ApiKeyRepository extends JpaRepository<ApiKey, Long> {
    Optional<ApiKey> findByApiKey(String apiKey);

    @Query("""
            select apiKey from ApiKey apiKey
            where lower(apiKey.userName) not like 'demo%'
              and lower(apiKey.userName) not like 'sample%'
              and lower(apiKey.apiKey) not like 'demo%'
              and lower(apiKey.apiKey) not like 'sample%'
            """)
    List<ApiKey> findAllRealKeys();

    @Query("""
            select apiKey from ApiKey apiKey
            where (
                :search = ''
                or lower(apiKey.userName) like concat('%', :search, '%')
                or lower(apiKey.apiKey) like concat('%', :search, '%')
            )
            and lower(apiKey.userName) not like 'demo%'
            and lower(apiKey.userName) not like 'sample%'
            and lower(apiKey.apiKey) not like 'demo%'
            and lower(apiKey.apiKey) not like 'sample%'
            """)
```

Service Layer

```java
@Service
public class ApiKeyService {

    private final ApiKeyRepository apiKeyRepository;
    private final String defaultAlgorithm;
    private final ConcurrentHashMap<String, ApiKey> apiKeyCache = new ConcurrentHashMap<>();
    private final ConcurrentHashMap<String, ApiKeyDelta> pendingCounterDeltas = new ConcurrentHashMap<>();

    public ApiKeyService(
            ApiKeyRepository apiKeyRepository,
            @Value("${ratelimiter.default-algorithm:SLIDING_WINDOW}") String defaultAlgorithm
    ) {
        this.apiKeyRepository = apiKeyRepository;
        this.defaultAlgorithm = normalizeOrDefault(defaultAlgorithm, "SLIDING_WINDOW");
    }

    @PostConstruct
    public void warmCache() {
        refreshCache();
    }

    public ApiKey createApiKey(String userName, Integer rateLimit, Integer windowSeconds) {
        if (userName == null || userName.trim().isEmpty()) throw new IllegalArgumentException("userName is required");
        if (rateLimit == null || rateLimit <= 0) throw new IllegalArgumentException("rateLimit must be > 0");
        if (windowSeconds == null || windowSeconds <= 0) throw new IllegalArgumentException("windowSeconds must be > 0");
```

Contoller Layer

```java
@RestController
@RequestMapping("/api")
public class ApiKeyController {

    private final ApiKeyService apiKeyService;
    private final RequestStatsService requestStatsService;
    private final DistributedRateLimiterService distributedRateLimiterService;
    private final DecisionAuditService decisionAuditService;
    private final long dashboardCacheTtlMs;
    private final ConcurrentHashMap<String, CachedDashboardView> dashboardCache = new ConcurrentHashMap<>();

    public ApiKeyController(
            ApiKeyService apiKeyService,
            RequestStatsService requestStatsService,
            DistributedRateLimiterService distributedRateLimiterService,
            DecisionAuditService decisionAuditService,
            @Value("${ui.dashboard-cache-ttl-ms:500}") long dashboardCacheTtlMs
    ) {
        this.apiKeyService = apiKeyService;
        this.requestStatsService = requestStatsService;
        this.distributedRateLimiterService = distributedRateLimiterService;
        this.decisionAuditService = decisionAuditService;
        this.dashboardCacheTtlMs = Math.max(100L, dashboardCacheTtlMs);
    }
```

Table_Box.jsx

```jsx
import { useState, useEffect } from "react";
import { Plus, Copy, Search } from "lucide-react";
import { apiUrl } from "../apiBase.js";
import { buildAuthHeaders } from "../auth.js";
import "../Styles/Table_Box.css";

function ApiTable({
  dashboardData,
  loading,
  refreshing,
  defaults,
  onDashboardRefresh,
  tableQuery,
  onTableQueryChange,
}) {
  const defaultRateLimit = defaults.rateLimit;
  const defaultWindowSeconds = defaults.windowSeconds;
  const [isCreateModalOpen, setIsCreateModalOpen] = useState(false);
  const [formState, setFormState] = useState({
    userName: "",
    rateLimit: defaultRateLimit,
    windowSeconds: defaultWindowSeconds,
  });
  const [createError, setCreateError] = useState("");
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [copiedText, setCopiedText] = useState("");
  const [successModal, setSuccessModal] = useState({
```

LoginPage.jsx

```jsx
import { useState } from "react";
import { Eye, EyeOff, AlertCircle } from "lucide-react";
import { setFrontendAuthenticated } from "../auth.js";
import { apiUrl } from "../apiBase.js";
import "../Styles/LoginPage.css";

function LoginPage() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [showPassword, setShowPassword] = useState(false);

  const handleSubmit = async (event) => {
    event.preventDefault();
    if (isSubmitting) return;

    setError("");
    setIsSubmitting(true);

    try {
      const response = await fetch(apiUrl("/api/auth/login"), {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
```